WEEK 2 · DAY 1 · RULE BOOK

Take It Apart: Trend, Season and Noise

Follow it step by step. Every common error and its fix is here.

Your goal today: take three different series apart with `seasonal_decompose`, choose the right **period** and **model** each time, and read the **residual** to find a hidden fault. Earn **13 green ✅ ticks** (12 tasks + the 🌶️ bonus). Time: about 60 minutes. Keep this page open next to Colab.

Before you start (2 minutes)

- ☐ Open `W2D1-exercise` in Colab, then **File → Save a copy in Drive**. Work in the new tab.
- ☐ Run **STEP 0** and wait for ✅ `Ready!`
- ☐ Run the cell that loads `seasonal_decompose` and `STL`.

The 6 golden rules of decomposition

#	Rule	Why
1	**observed = trend + seasonal + residual**	That's the additive model. The three parts add back to the original exactly.
2	**The period comes from the process**	Ask: "what makes this repeat, and how many rows until it repeats?" Hourly + daily eclipse → 24. Daily + working week → 7. Monthly + rainy season → 12.
3	**Swing grows with the level? Multiply.**	Use `model="multiplicative"`, or take `np.log()` first and stay additive.
4	**Three full cycles minimum**	Fewer, and the "season" might be one lucky year.
5	**Look for trouble in the residual**	Trend and season are explained. The leftovers are the surprises.
6	**One disaster or a moving season? Use `STL(robust=True)`**	It stops one bad month from spoiling every month like it.

The recipe

```
from statsmodels.tsa.seasonal import seasonal_decompose, STL

series = df["column"]                                    # a DatetimeIndex with .asfreq()
set, no NaN inside
parts = seasonal_decompose(series, model="additive", period=12)
parts.trend, parts.seasonal, parts.resid                 # the three parts
fig = parts.plot(); fig.set_size_inches(11, 7); plt.show()

robust = STL(series, period=12, robust=True).fit()       # the smarter, outlier-proof
version
```

The exercise, step by step

Task	What you do	The tool you need
A1	Frequency code for month start	`"MS"`
A2	Count full rainy seasons	`len(kam) // 12`
A3	Period for monthly data with a yearly season	a number
A4	Prove trend + seasonal + residual = observed	`parts.trend + parts.seasonal + parts.resid`
A5	Month the season hits hardest	`parts.seasonal.idxmin().month`
B1	Period for daily NOC tickets	a number
B2	How many tickets below normal on Sunday	`weekly.loc["Sunday"]`, rounded to 1 decimal
B3	Which period leaves more noise	compare the two printed numbers
B4	Additive or multiplicative for column 2	look at the swing
B5	The log trick	`np.log(demo["multiplicative"])`
B6	Time of the biggest leftover in August	`aug_parts.resid.abs().idxmax()`
B7	The two battery dip months	a list like `[3, 9]`
🌶️ **Bonus**	Robust STL vs one disaster August	`robust=True`

When things go wrong: error → fix

You see	It means	Fix
`ValueError: x must have 2 complete cycles requires 24 observations`	Not enough rows for that period	Use more data, or check the period is right
`ValueError: This function does not handle missing values`	There's a NaN inside the series	Find it with `.isna().sum()` ; fill or trim it first
`ValueError: Multiplicative seasonality is not appropriate for zero and negative values`	Multiplicative needs all values above zero	Use additive, or check for zeros
`ValueError: You must specify a period…`	No `period=` and no frequency on the index	Add `period=` (or `.asfreq()` the series)
`AssertionError: ✍️ Do A1 first…`	A blank above is still empty	Go back up and fill it in
Trend panel is empty at both ends	Normal! A 12-month moving average needs 6 months on each side	Nothing to fix
Picture shows twice	The plot was returned **and** shown	Write `fig = parts.plot()` then `plt.show()`
`NameError: name 'seasonal_decompose' is not defined`	The import cell wasn't run (or Colab restarted)	Run STEP 0 and the import cell

Stuck? Climb the ladder

1. **Read the ❌ hint.**
2. **Check the error table above.**
3. **Ask your neighbour.**
4. **Ask the AI button** to explain the error. Paste your code and the error, never real data.
5. **Raise your hand.**

🏁 **Finished?** Run `scoreboard()` . Then help someone near you choose their period. It's the question everyone gets wrong at least once.